HACKATHON PROTOTYPE REPORT

BhoomiSetu

Intelligent Land Record Digitization and Validation System

Turning legacy land records into searchable, validated, and tamper-evident digital records.

Report type	Technical project report
Build stage	Functional hackathon prototype
Report date	09 September 2026
Live prototype	bhoomisetu-land-records.techsawyer250.chatgpt.site

01 / OVERVIEW

Executive summary

BhoomiSetu is a browser-based land-record digitization and verification prototype designed for a 12-hour hackathon build. It demonstrates a complete, judge-ready path from a scanned record to structured fields, human validation, a deterministic cryptographic fingerprint, and two proof options: gasless wallet attestation or Polygon Amoy anchoring.

Core outcome

The prototype proves that legacy records can be converted into reviewable digital records and that any later change can be detected by recomputing the approved fingerprint. The design keeps names, documents, and parcel details off-chain.

Project objectives

- **Digitize:** accept legacy record scans and extract structured information using OCR.
- **Validate:** require human confirmation and run completeness, format, area, date, and duplicate-parcel checks.
- **Search:** provide a usable registry view across record ID, owner, survey number, village, and record number.
- **Protect integrity:** canonicalize each approved record and calculate a SHA-256 fingerprint.
- **Verify trust:** support wallet-signed attestations and optional Polygon Amoy transaction anchoring.

Current prototype status

Capability	Status	Notes
Interface and workflow	Implemented	End-to-end modal workflow and searchable registry.
OCR	Implemented	Browser-side English and English + Kannada recognition with preprocessing and confidence signals.
Validation	Implemented	Human review plus duplicate-parcel and required-field checks.
Gasless wallet proof	Implemented	MetaMask personal signature and signer recovery.
Polygon Amoy anchor	Implemented with dependency	Requires MetaMask and test POL for network gas.
MongoDB persistence	Not yet integrated	Current records and new proofs are held in browser memory for the session.
Merkle batch anchoring	Designed, not built	Recommended production evolution to amortize gas across many records.

02 / CONTEXT

Problem definition and scope

Legacy land records are often stored as scans or paper documents. They are difficult to search, prone to transcription errors, and hard to verify after digitization. A trustworthy system must preserve the original evidence, support human review, and make unauthorized changes detectable without exposing personal data on a public ledger.

In scope for the prototype

- PNG and JPEG upload up to 12 MB.
- Image normalization using grayscale, contrast enhancement, resizing, and percentile-based tonal stretching.
- Tesseract.js OCR with English and optional Kannada language selection.
- Structured extraction for owner, survey number, village, area, record number, and issue date.
- Field-level confidence cues and editable human review.
- Duplicate parcel warning using village and survey number.
- Deterministic SHA-256 fingerprint generation.
- Gasless wallet signature and optional Polygon Amoy anchoring.
- Integrity verification by recomputation and proof comparison.

Explicitly outside the current build

- Production-grade identity, authorization roles, and audit administration.
- PDF ingestion, handwriting recognition, layout-aware OCR, and large batch uploads.
- Authoritative cadastral or government-registry integration.
- Durable MongoDB persistence, file storage, backups, and migration tooling.
- Production key custody, relayer service, paymaster, or automatic Merkle batch anchoring.
- Legal certification or replacement of official land-title processes.

Scope principle

The prototype favors a reliable, explainable demonstration over an ambitious but incomplete platform. It is a proof of workflow and integrity architecture, not a production land registry.

03 / DESIGN

Solution architecture

The current application is a client-heavy web prototype. OCR, normalization, hashing, wallet requests, and most validation run in the browser. This protects uploaded documents from unnecessary transmission and keeps the demonstration deployable without a specialized processing backend.

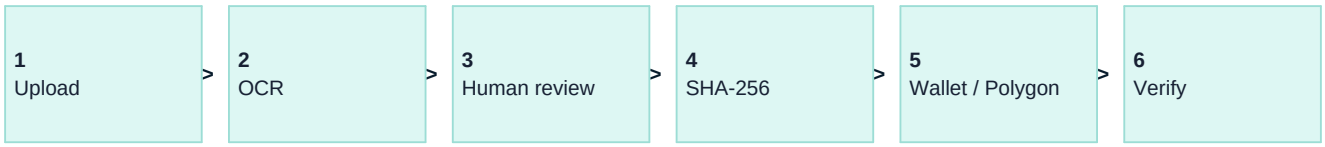

Figure 1. Current end-to-end record workflow.

Logical components

Layer	Technology	Responsibility
Presentation layer	Vinext / React interface	Registry, upload, review, proof selection, and verification views.
OCR engine	Tesseract.js	Runs recognition locally in the browser; reports progress and overall confidence.
Image preparation	Canvas API	Resizes large scans, applies grayscale and contrast, and normalizes tonal range.
Validation engine	Client-side rules	Required fields, positive area, date/identifier handling, and duplicate parcel warning.
Integrity engine	Web Crypto SHA-256	Produces a stable fingerprint from canonical record fields.
Wallet proof	MetaMask personal_sign	Signs the record ID and fingerprint without broadcasting a transaction.
Blockchain proof	Polygon Amoy RPC	Stores the fingerprint in transaction input and reads it back after confirmation.
Persistence target	MongoDB	Planned store for records, OCR metadata, signatures, proof state, and audit history.

Deployment

The prototype is packaged as a Vinext application and privately published using OpenAI Sites. The current implementation contains no production database secret, private blockchain key, or server-side custody component.

04 / INTELLIGENCE

OCR and validation pipeline

OCR is intentionally treated as an assistive extraction step rather than an authority. The application exposes confidence and raw OCR output, then requires a reviewer to confirm every field before approval.

OCR processing sequence

Stage	Behavior
1. Input gate	Accept PNG/JPEG only; reject files over 12 MB.
2. Preprocess	Respect image orientation, resize toward a 2400 px working width, convert to grayscale, and increase contrast.
3. Normalize	Use a pixel histogram to clip extreme dark/light percentiles and stretch usable tonal range.
4. Recognize	Run Tesseract.js using English or English + Kannada language data.
5. Retry	When enhanced-image confidence is below 68%, compare against recognition on the original image.
6. Parse	Apply label-aware and regular-expression extraction to populate six target fields.
7. Score	Derive field confidence from overall OCR confidence and whether a usable field was extracted.
8. Review	Display source and fields side by side; allow corrections before approval.

Validation rules demonstrated

- All six record fields must be populated.
- Area must parse to a number greater than zero.
- Dates and record identifiers are normalized during parsing.
- A matching survey number and village triggers a possible parcel-conflict warning.
- Approval is disabled while required data is missing or a duplicate conflict remains unresolved.

Reliability boundary

Confidence percentages are review aids, not legal certainty. Handwriting, damaged scans, mixed layouts, and administrative abbreviations require additional models, training data, or manual entry.

05 / INTEGRITY

Trust, wallet, and blockchain model

BhoomiSetu uses data minimization: only a cryptographic fingerprint is signed or anchored. The original scan and personal record fields remain off-chain. This avoids public exposure, reduces transaction size, and permits controlled corrections while preserving a verifiable history.

Deterministic fingerprint

Before hashing, values are normalized: names and villages are trimmed and lowercased, survey and record numbers are uppercased, area is fixed to two decimals, and the date is stored in ISO form. The canonical JSON payload is then hashed with SHA-256.

Gasless wallet attestation

- MetaMask signs a human-readable message containing the record ID, SHA-256 fingerprint, and approval purpose.
- No chain switch, POL balance, or transaction fee is required.
- Verification recovers the signer address from the message and signature and checks that the record fingerprint is unchanged.
- The signature is not stored by MetaMask and is not discoverable on-chain; BhoomiSetu must persist it.

Polygon Amoy anchoring

- The wallet switches to Polygon Amoy, chain ID 80002.
- A zero-value self-transaction carries the fingerprint in its input data.
- The application waits for a successful receipt, stores the transaction hash, and reads the transaction input back during verification.
- This path is genuinely on-chain but cannot operate from an unfunded account because every public blockchain transaction consumes gas.

Important distinction

Wallet-attested means cryptographically approved by a wallet. Polygon-anchored means the fingerprint was included in a confirmed public-chain transaction. The interface labels these states separately.

06 / DATA

Persistence and proposed MongoDB model

The deployed prototype currently uses in-memory React state and seeded demonstration records. A page refresh discards newly created records and signatures. MongoDB integration is therefore the highest-priority step before multi-session testing or any operational pilot.

Recommended record document

Field group	Contents
_id / recordId	Stable internal and human-readable identifier.
source	Object storage key, MIME type, checksum, and upload metadata; do not store large scans directly in the record document.
fields	Owner, survey, village, area, record number, and issue date.
ocr	Language, raw text reference, overall confidence, per-field confidence, preprocessing version, and timestamp.
validation	Reviewer, rule results, conflicts, approval state, and approved timestamp.
integrity	Canonicalization version and approved SHA-256 fingerprint.
walletProof	Signed message, signature, signer address, signing scheme, and signed timestamp.
chainProof	Network, chain ID, transaction hash, block number, anchored fingerprint, and confirmation state.
history	Append-only version references for corrections and superseded approvals.

Minimum indexes

- Unique index on recordId.
- Compound index on village and survey number for conflict detection.
- Search indexes for owner, record number, and normalized parcel identifiers.
- Index on integrity.fingerprint for proof lookup and duplicate detection.
- Indexes on validation.state and chainProof.confirmationState for work queues.

Security controls required

- Keep database credentials server-side; never expose MongoDB credentials in browser JavaScript.
- Use role-based access, encryption in transit, encrypted backups, and a restricted network allowlist.
- Store original documents in controlled object storage and retain immutable checksums.
- Record every correction as a new version instead of silently overwriting an approved snapshot.

07 / SCALE

Recommended Merkle batch evolution

For production, individual fingerprints should be grouped into a Merkle tree and only the Merkle root anchored on Polygon. Each record retains a compact inclusion proof. This preserves public verifiability while spreading one transaction fee across hundreds or thousands of records.

Step	Production behavior	User experience
1	Reviewer approves and signs the record fingerprint.	Immediate confirmation.
2	Backend stores the signed record and places its fingerprint in an anchor queue.	Status: Wallet attested - queued.
3	Scheduled worker creates a deterministic Merkle tree from queued fingerprints.	No user action.
4	A controlled relayer submits one root to an anchoring contract.	No user-held gas required.
5	Backend saves batch ID, root, transaction, leaf position, and sibling path.	Status: Blockchain anchored.
6	Verifier recomputes the leaf and Merkle path, then compares the root with Polygon.	One-click proof result.

Why this is viable

Merkle batching does not remove network fees; it amortizes them. A sponsor or government operator funds one transaction per batch while end users experience a gasless workflow.

Recommended contract surface

`anchorRoot(bytes32 root, uint256 batchId)` should emit an event containing the root, batch ID, submitter, and block timestamp. Full documents, names, and parcel metadata should never be contract storage.

Operational safeguards

Control	Purpose
Deterministic ordering	Create each batch from an immutable ordered list and retain the algorithm version.
Duplicate rejection	Prevent the same fingerprint from appearing twice within one batch.
Controlled submission	Use an allowlisted relayer and multisignature administration.
Stable retries	Retry failed submission without changing the original batch composition.
Portable proof	Export inclusion proofs so verification is not dependent on the application UI.

08 / DEMO

Demonstration and verification plan

Recommended three-minute demonstration

Time	Moment	Action
0:00-0:25	Problem	Show an unstructured legacy record and explain why search and trust are difficult.
0:25-1:05	Digitize	Load the sample or upload a scan; highlight OCR progress, extracted fields, and confidence.
1:05-1:35	Validate	Correct one field and show rule checks plus duplicate-parcel detection.
1:35-2:10	Protect	Approve the record, generate the deterministic fingerprint, and sign the gasless wallet attestation.
2:10-2:40	Prove	Verify the signature, change one character in the owner field, and show integrity failure.
2:40-3:00	Scale story	Explain optional Polygon anchoring today and Merkle batch anchoring for production.

Current verification evidence

- Production build completes successfully with Vinext.
- OCR reports progress and includes a low-confidence fallback pass.
- Approved record hashes are generated through the browser Web Crypto API.
- Wallet signatures are recoverable through the injected wallet provider.
- Polygon receipts are checked for success and transaction input can be read back from Amoy.
- Tamper testing is exposed directly in the proof view by editing the owner field and recomputing the fingerprint.

Tests still required before pilot use

- A representative corpus across scan quality, forms, languages, and administrative eras.
- Measured field-level precision, recall, correction rate, and reviewer time.
- API, persistence, authorization, concurrency, and audit-history tests.
- Threat modeling for replay, signature substitution, database tampering, compromised relayers, and document replacement.
- Accessibility, mobile layout, browser compatibility, and wallet-provider interoperability testing.

09 / DELIVERY

Risks, limitations, and roadmap

Risk	Severity	Impact	Mitigation
OCR variability	High	Scans and layouts may differ sharply from the demonstration sample.	Build a labeled corpus; add layout detection and document-type templates.
No durable persistence	Critical	New records disappear on refresh.	Implement server-side APIs, MongoDB, and controlled document storage first.
Wallet proof loss	High	Gasless signatures are unrecoverable if the app does not persist them.	Persist full proof payload immediately and support proof export.
Faucet / gas dependency	High	Polygon test anchoring can fail for unfunded wallets.	Use a funded relayer and Merkle batches; retain gasless signing for users.
Legal authority	Critical	Cryptographic integrity does not establish legal title or correct source data.	Integrate authorized registries, reviewer identity, and jurisdictional process controls.
Privacy	High	Land records contain sensitive personal information.	Minimize collection, enforce access policy, encrypt storage, and keep PII off-chain.

Phased roadmap

Phase	Deliverables
Phase 1 - Durable prototype	MongoDB API, object storage, proof persistence, record retrieval, and server-side validation.
Phase 2 - Trust hardening	EIP-712 typed signatures, replay protection, reviewer roles, immutable version history, and proof export.
Phase 3 - Batch anchoring	Merkle tree worker, anchoring contract, funded relayer, inclusion proof API, and automatic confirmation tracking.
Phase 4 - OCR evaluation	Document templates, dataset benchmarks, handwriting strategy, multilingual tuning, and correction analytics.
Phase 5 - Pilot readiness	Security review, disaster recovery, accessibility testing, monitoring, legal workflow mapping, and controlled pilot.

Immediate next decision

Prioritize MongoDB persistence before adding more blockchain complexity. Without durable storage, neither wallet signatures nor Merkle inclusion proofs can be retrieved after the browser session ends.

10 / SUMMARY

Conclusion

BhoomiSetu successfully demonstrates the essential hackathon story: extract a legacy record, make uncertainty visible, require a human decision, fingerprint the approved snapshot, and detect later changes. The result is coherent, explainable, and usable as a foundation for a more rigorous land-record workflow.

The most important architectural conclusion is that blockchain should protect integrity, not store land records. Gasless wallet attestation provides immediate authorization; Polygon anchoring provides public timestamped evidence; and a future Merkle batching layer can combine the two into a seamless and economical production experience.

Final assessment

The prototype is ready for hackathon demonstration. It is not yet ready for real land administration because durable persistence, authoritative identity, representative OCR evaluation, security controls, and legal-process integration remain unfinished.

Technology summary

Area	Current selection
Frontend	React 19 with Vinext and Tailwind CSS
OCR	Tesseract.js 6 with Canvas preprocessing
Hashing	SHA-256 through Web Crypto
Wallet proof	MetaMask personal signing and signer recovery
Blockchain	Polygon Amoy, chain ID 80002
Database target	MongoDB - planned, not integrated in the current prototype
Hosting	Private OpenAI Sites deployment

References

Polygon account abstraction: docs.polygon.technology/pos/concepts/transactions/eip-4337
Polygon meta-transactions: docs.polygon.technology/pos/concepts/transactions/meta-transactions
Tesseract.js: github.com/naptha/tesseract.js

Prepared from the implemented BhoomiSetu prototype source and deployment state as of 09 September 2026.